Intelligent Placement Assistant:

A Retrieval-Augmented Generation (RAG) System for Campus Recruitment Support

Project Report (Prototype Stage) for UCS503P

Submitted to: **Dr. Paramveer Sandhu**

Oshish Madhan (omadhan_be24@thapar.edu)
Department of Computer Science and Engineering
Thapar Institute of Engineering and Technology

Abstract — *This report presents the prototype implementation of the Intelligent Placement Assistant for course UCS503P. Grounded strictly upon 9,302 real historical campus interview questions across 212 recruiting companies and 16 verified candidate placement experiences, the system integrates atomic document chunking, 384-dimensional dense sentence embeddings via all-MiniLM-L6-v2, persistent vector storage with ChromaDB, dual-mode retrieval (cosine semantic similarity and complete keyword scanning), grounded generation via Gemini 3.5 Flash with source citations, an in-memory analytics engine, and a full React/TypeScript web interface. The prototype demonstrates a 100% pass rate across 7 test suites (47 test cases) and sub-second retrieval latency.*

1. Executive Summary & Prototype Status

Campus placement preparation at Thapar Institute requires candidates to navigate extensive interview records across hundreds of recruiters. Generic conversational LLMs frequently hallucinate fake questions. This project delivers a production-grade RAG application delivering grounded, company-attributed question guidance.

Key Prototype Achievements:

- **Real-World Dataset Ingestion:** 9,302 questions across 317 company-role records and 212 visiting campus companies processed with deterministic IDs ({record_id}_q{index}).
- **ChromaDB Vector Store:** Dense 384-dimensional embeddings (sentence-transformers/all-MiniLM-L6-v2) indexed across 73 batches into local persistent disk storage with cosine distance metric.
- **Dual Retrieval Engine:** Combines dense vector similarity search with exhaustive keyword/metadata filtering (zero truncation for broad queries like "all HashMap questions").
- **Grounded Gemini LLM:** Powered by gemini-3.5-flash with strict prompt instructions enforcing source attribution and providing a zero-crash offline local fallback mode.
- **In-Memory Analytics:** Sub-millisecond pre-computed aggregations for topic frequency, company hiring volume, and preparation categories.
- **Full-Stack Application:** High-performance FastAPI backend (port 8000) with a premium 3-page dark-themed React + TypeScript UI (port 5173).
- **Automated Testing:** 47 automated tests spanning 7 test suites passing with a 100% success rate.

2. Dataset Analysis & Preprocessing

The prototype is grounded in the proprietary TietPrep campus placement dataset:

- **Total Questions:** 9,302 individual interview questions.

• **Recruiting Companies:** 212 organizations (e.g. American Express: 321 Qs, Deloitte: 225 Qs, Blackrock: 146 Qs, IBM: 146 Qs, ProcDna: 138 Qs).

• **Category Split:** Technical: 5,108 questions (54.9%), Additional Practice: 2,536 questions (27.3%), HR / Behavioral: 1,658 questions (17.8%).

• **Graduation Cohorts:** Batch 2026 (6,099 Qs), Batch 2027 (2,904 Qs), Batch 2028 (299 Qs).

• **Verified Candidate Experiences:** 16 on-campus placement reports with verified offers.

3. System Architecture & Components

The architecture follows a decoupled, service-oriented design:

1. **Document Preparation (prepare_documents.py):** Atomic question modeling, deterministic ID generation, flat ChromaDB-compatible metadata.

2. **Embedding Engine (embeddings.py):** In-memory cached SentenceTransformers producing normalized 384-dim vectors.

3. **Vector Store (index_chroma.py):** ChromaDB persistent storage at code/backend/data/chroma_db with idempotent batch upserts.

4. **Retrieval Service (retriever.py):** Cosine semantic search (similarity score = 1 - cosine_distance) and full-scan pagination.

5. **LLM Generation (generator.py):** Grounded Gemini prompt construction, source attribution pills, and offline resilience.

6. **Analytics Engine (analytics.py):** In-memory cached statistics for sub-millisecond responses.

7. **FastAPI Backend (main.py):** REST endpoints (/api/health, /api/chat, /api/questions, /api/filters, /api/analytics).

8. **React + TypeScript Frontend:** Three dedicated views (AI Assistant, Question Library, Interview Insights).

4. Empirical Verification & Test Results

Test Module	Scope	Cases	Result
test_prepare_documents.py	Document chunking, schema validation, ID stability	9	PASSED
test_embeddings.py	Vector dimensions (384), normalization, batching	10	PASSED
test_index_chroma.py	ChromaDB collection setup, idempotent upserts	5	PASSED
test_retriever.py	Semantic similarity scoring, keyword search	6	PASSED
test_generator.py	Grounding prompts, fallback mode, key redaction	3	PASSED
test_analytics.py	Topic distributions, company ranking, cache sanity	3	PASSED
test_api.py	FastAPI REST endpoint integration testing	11	PASSED
Total Suite	**Full End-to-End System Verification**	**47**	**100% PASSED**

5. Performance Benchmarks (Apple Silicon M4)

• **Vector Similarity Query Latency:** 12ms – 18ms per top-5 semantic query.

• **Full Dataset Keyword Scan:** 20ms – 35ms across 9,302 items.

• **Analytics Dashboard Latency:** 2ms – 4ms (in-memory cached).

• **End-to-End Gemini Generation:** 1.1s – 2.2s using gemini-3.5-flash.

• **Frontend Build Time:** 609ms producing an 81.5KB compressed production bundle.

6. Conclusion

The prototype phase of the Intelligent Placement Assistant successfully solves the problem of unstructured, scattered placement resources. Grounded on real institutional records from 212 recruiting organizations, the system combines precision vector search with state-of-the-art LLM synthesis, providing a verified, scalable platform for Thapar Institute students.